SESSION 1 OF 8

The New Engineering Stack

Copilot as a Development Partner

DAY 1 — Foundations of AI-Augmented Engineering

Duration: 90 Minutes | Theory • Demo • Hands-on

Anurag Kumar

WORKSHOP JOURNEY

4 Days × 8 Sessions × Moving Across the SDLC

DAY 1

Foundations

Sessions 1 & 2

How Copilot works

Prompt-driven coding

▶ YOU ARE HERE

DAY 2

AI Across SDLC

Sessions 3 & 4

Requirements analysis

Testing & QA

DAY 3

Advanced Usage

Sessions 5 & 6

Copilot CLI

Code review & docs

DAY 4

Future + Gov

Sessions 7 & 8

Agentic development

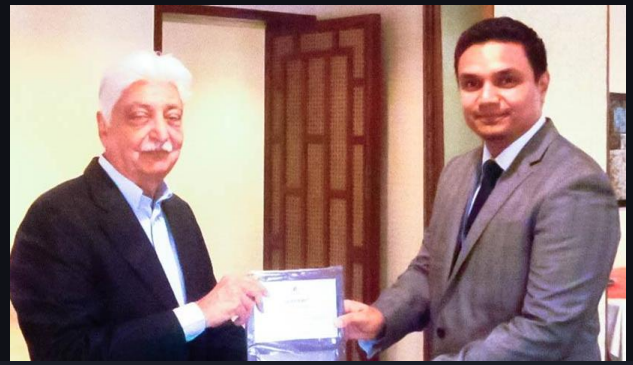
Security & governance

ABOUT YOUR TRAINER

Anurag Kumar

Founder, Prex Studio | GenAI SME, Ethan's Tech

- B.Tech, IIT Bombay | MBA, IIM Bangalore
- 14+ Years of Experience Building AI Solutions
  Wipro Global 100 Leadership Program | VP Marketing, Inviz AI
- Trained 5k professionals on GenAI in 2025
- International Conference Speaker on Generative AI
- Advisor AI Adoption for NSRCEL (IIM Bangalore) Incubation Start-Ups
- IIT Guwahati — Upskilling Faculty & Scholars (MeitY, Govt. of India)



SESSION GOAL

Reset your mental model

from ~~"AI autocomplete"~~

to **"AI engineering collaborator"**

By the end of this session, you will understand:

- How Copilot works under the hood
- What context sources it uses and how to maximize them
- Different interaction modes and when to use each

SESSION AGENDA

Use AI is a collaborator not just a glorified auto-complete

01 How Copilot Works

LLM foundations, token limits, context window

02 Context Sources

Current file, repo, comments, chat prompts

03 Interaction Modes

Inline, Chat, Copilot CLI

04 Configuration

VS Code settings, Business vs Individual

THE FUNDAMENTAL SHIFT

Programming vs. Artificial Intelligence

Traditional Programming

DETERMINISTIC

Rules + Data = Output

- You write explicit instructions
- Same input always gives same output
- Errors are bugs in your logic
- Computer does exactly what you say
- You must anticipate every case

Artificial Intelligence

PROBABILISTIC

Data + Patterns = Prediction

- Model learns from examples
- Same input can give different output
- Errors are confidence misjudgments
- Model predicts what's most likely
- It generalizes from patterns it's seen

When you use Copilot, you're working with prediction, not execution.

THE EVOLUTION

Language Models to Autonomous Agents

Each step made AI more useful

2020

Base LLM

Predicts next token based on training data. No instruction following.

GPT-3

2022

Instruction-Tuned

Trained to follow human instructions. Conversational ability.

ChatGPT

2024

Reasoning Model

Chain-of-thought thinking. Plans before answering.

o1, Claude 3.5

2025

Agentic AI

Takes multi-step actions. Uses tools. Self-corrects.

Copilot Agent Mode

THE INSPIRATION

AI is Inspired by Biology, Not Logic

What AI borrowed from the brain

Neurons firing together

Neural network layers processing data

Paying attention to what matters

Attention mechanism (Transformers)

Learning from experience

Training on billions of examples

Pattern recognition

Statistical pattern matching at scale

Why This Matters

AI doesn't "think" like a computer. It recognizes patterns like a brain.

This is why:

- It's creative but sometimes wrong
- Context changes everything
- It can generalize to new problems
- It "hallucinates" with confidence

THE ROLE EVOLUTION

How the Dev Role Has Changed

BEFORE AI

Code Writer

- Write every line manually
- Debug by reading stack traces
- Google + StackOverflow for answers
- Write docs after the code is done
- Manual test case creation
- Value = lines of code shipped

→

WITH AI

AI-Augmented Engineer

- Define intent, AI generates code
- Ask AI to diagnose and fix
- Conversation with the codebase
- Docs generated alongside code
- AI generates test suites from code
- Value = problems solved, quality shipped

YOUR SUPERPOWER

What You Bring That AI Cannot

AI is powerful. But it needs you.

Business Context

You understand why the code exists.
AI only sees how it works.

Architectural Judgment

Choosing between microservices and monolith isn't a pattern match.

Security & Ethics

You catch what AI misses: compliance rules, privacy laws, moral edge cases.

Stakeholder Communication

Translating technical decisions into business language.
AI can't sit in that meeting.

System Thinking

Understanding how 50 services interact at scale.
AI sees files, you see systems.

Creative Problem Solving

The novel solution that isn't in the training data.
That's uniquely human.

AI is a leverage. You are the engineer. The best results come when both work together.

HOW COPILOT WORKS

Under the Hood

Your Code & Context → Context Assembly → Large Language Model (LLM) → Code Suggestion

Key Insights

Token Limits

Copilot has a finite context window. It cannot read your entire repo at once — it prioritizes the most relevant files.

Context is King

The quality of Copilot's output is directly proportional to the quality of context you provide. Better context = better code.

Not Deterministic

Same prompt can produce different outputs. Copilot is probabilistic — treat suggestions as starting points, not final answers.

CONTEXT SOURCES

What Copilot Sees When You Code

Chat Prompts

Explicit instructions via Copilot Chat

Comments & Docstrings

Natural language context in code

Current File

Active editor tab — imports, functions, classes

Open Tabs & Repo Files

Neighboring files, project structure, dependencies

Maximizing Context Quality

- Use descriptive file and function names
- Keep related code in nearby files
- Write clear comments before complex logic
- Open relevant files in VS Code tabs
- Use #codebase for repo-wide questions
- Structure your project consistently

CONTEXT DEEP DIVE

How Copilot Prioritizes What It Reads

Context Source	What Copilot Reads	How to Optimize
Current File	All code above cursor, imports, class signatures, comments	Place cursor strategically; write comments above target code
Open Tabs	Function signatures, class names, types from other open files	Open related files (DTOs, interfaces, tests) before prompting
Repository	Project structure, pom.xml/build files, README, nearby files	Maintain clean project structure; keep README updated
Comments	Inline comments, Javadoc, TODO markers, natural language	Write intent-rich comments that describe WHAT and WHY
Chat Prompts	Your explicit question + selected code + conversation history	Be specific; include constraints, format, and context in prompts

INTERACTION MODES

Four Ways to Work with Copilot

Inline Completion

Predictive suggestions as you type. Copilot reads your current context and offers code completions inline.

Best for: Writing new code, boilerplate, repetitive patterns

Copilot Chat

Conversational AI within VS Code. Ask questions, request explanations, or generate code via dialogue.

Best for: Understanding code, debugging, generating with constraints

Repo Chat

Query across the entire codebase. Copilot indexes your repository to answer project-wide questions.

Best for: Finding usages, understanding architecture, cross-file analysis

Copilot CLI

AI assistance in the terminal. Explain commands, generate scripts, and automate shell tasks.

Best for: DevOps tasks, build scripts, unfamiliar CLI commands

INLINE vs CHAT

Reactive vs. Proactive AI Assistance

Inline Completion

REACTIVE

You write → Copilot suggests

- Triggered by your keystrokes
- Best for boilerplate and patterns
- Tab to accept, Esc to dismiss
- Low friction, high speed
- Limited to current file context

Copilot Chat

PROACTIVE

You ask → Copilot answers

- Conversational, multi-turn dialogue
- Best for complex tasks and explanations
- Can reference selected code
- Higher precision, more control
- Can use #codebase for repo-wide scope

CONFIGURATION

Setting Up Copilot for Maximum Effectiveness

VS Code Settings

- Enable/disable per language or file type
- Configure inline suggestion trigger behavior

Keyboard shortcuts:

Tab	— Accept suggestion
Esc	— Dismiss suggestion
Alt+]	— Next suggestion
Ctrl+Alt+I	— Open Copilot Chat

Business vs. Individual

Copilot Business:

- Organization-managed policies
- Code is NOT retained for training
- Admin controls and audit logging
- IP indemnification included

Your code stays your code.

USE CASES

Real-World Scenarios at FIS

NEW DEVELOPER

Codebase Onboarding

Ask Copilot to explain an unfamiliar Java service class, getting an instant onboarding summary of business logic, dependencies, and key patterns.

DEVELOPER

Legacy Code Understanding

Use Copilot Chat to understand a complex legacy method before modifying it. Avoid reading hundreds of lines manually by asking for a structured summary.

FUNCTIONAL ANALYST

API Documentation

Generate high-level documentation for an existing API so analysts can understand the system without reading Java code directly.

QA ENGINEER

Failure Point Analysis

Ask Copilot to identify potential failure points and edge cases in a service method before writing test scenarios.

HANDS-ON TIME

Guided Practice

Tools: VS Code + GitHub Copilot Chat + Java Transaction Service

EXERCISE 01

Explain Existing Code

What to do:

Open the pre-built TransactionService Java class. Use Copilot Chat to ask it to explain the codebase as if you're a new team member.

Try this prompt:

```
Explain this Java service like I am a new developer
joining the team. Highlight business logic and
potential failure points.
```

Pro Tip: Select the entire class before asking in Chat. This gives Copilot the full context to work with.

EXERCISE 02

Generate Documentation

What to do:

Use inline comments and Copilot Chat to generate comprehensive Javadoc documentation for all public methods in the service class. [TransferService.java]

Try this prompt:

```
Generate Javadoc for all public methods in this
class. Include parameter descriptions and return
values.
```

Pro Tip: Try generating docs with inline comments (type /** above a method) vs using Chat. Compare the results.

EXERCISE 03

Refactor Logic

What to do:

Find the most complex method in the TransactionService. Ask Copilot to break it down into smaller, more readable helper methods.

Try this prompt:

```
Refactor this method into smaller helper methods
with clear single responsibilities.
```

Pro Tip: After refactoring, ask Copilot: “Does this refactored version preserve the original behavior?”

EXERCISE 04

Identify Edge Cases

What to do:

Ask Copilot to think like a QA engineer and identify all possible edge cases and failure scenarios in the fund transfer method. [FraudCheckService.java]

Try this prompt:

```
What are the edge cases and potential failure
scenarios for this fund transfer method?
```

Pro Tip: Follow up with: “Which of these edge cases would be most critical in a fintech production environment?”

EXERCISE 05

Compare Interaction Modes

What to do:

Ask the same question using three modes: (1) inline comment, (2) Copilot Chat, (3) #codebase. Compare the depth and accuracy of each.

Try this prompt:

```
#codebase How is the TransactionService used
across this project?
```

Pro Tip: Notice how #codebase provides project-wide context that inline and basic Chat cannot access.

PROMPT REFERENCE

Session 1 — All Key Prompts

1. Explain this Java service like I am a new developer joining the team. Highlight business logic and potential failure points.

2. Generate Javadoc for all public methods in this class. Include parameter descriptions and return values.

3. Refactor this method into smaller helper methods with clear single responsibilities.

4. What are the edge cases and potential failure scenarios for this fund transfer method?

5. #codebase How is the TransactionService used across this project?

CORE CONCEPT

Copilot as a Codebase Conversationalist

Developers should start talking to the codebase, not just writing code.

Before: Read code → Understand → Write

Now: Ask Copilot → Understand → Iterate

DISCUSSION & WRAP-UP

Reflect, Share, and Look Ahead

Reflection Questions

- What surprised you about how Copilot uses context?
- Which interaction mode felt most natural for your workflow?
- Where do you see the biggest productivity win in your daily work?
- What concerns do you have about relying on AI suggestions?

UP NEXT

Session 2

Writing Better Code Faster with Copilot

Prompt-Driven Development